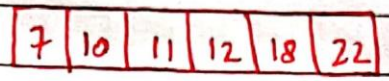
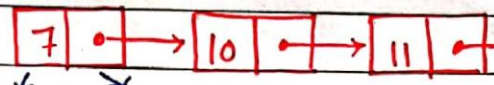
URBAN EDGE

Introduction to Linked Lists

Linked lists are similar to arrays (Linear data structures)



⇒ In Arrays elements are stored in contiguous memory locations



⇒ In linked lists, elements are stored in non contiguous memory locations

Why Linked Lists?

Memory and the capacity of an array remains fixed.
In case of linked lists, we can keep adding and removing elements without any capacity constraints

Drawbacks of Linked lists

→ Extra memory space for pointers is required (for every node 1 pointer is needed)
→ Random access not allowed as elements are not stored in contiguous memory locations.

Implementation

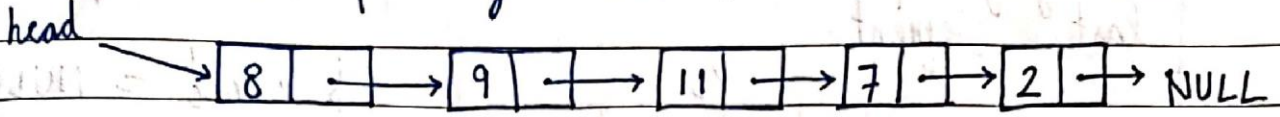
Linked list can be implemented using a structure in C language

```
Struct Node {
    int data;
    Struct Node * next;
};
```

⇒ Self refencing Structure

Deletion in a Linked List

Consider the following Linked List

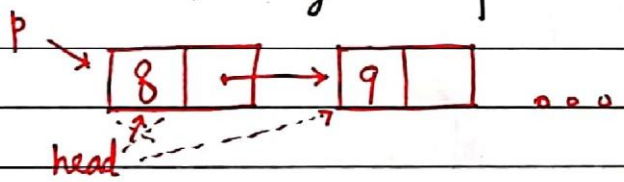


Deletion can be done for the following cases:

1. Deleting the first Node
2. Deleting the node at an index
3. Deleting the last Node
4. Deleting the first node with a given value.

The deletion just like insertion is done by rewiring the pointer connections, the only caveat being: we need to free the memory of the deleted node using free().

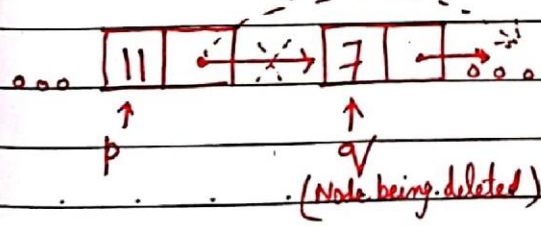
Case 1: Deleting the first node




```
Struct Node * p = head
head = head → next;
free(p);
```

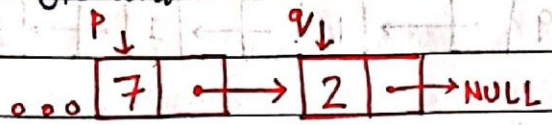
Case 2: Deleting the node at an index

for deleting a given node, we first bring a temporary pointer p before element to be deleted and q on the element being deleted



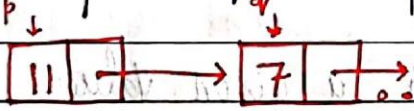

```
p → next = q → next;
free(q);
```

Case 3: Deleting the last Node
Last node can be deleted just like Case 2 by bringing p on second last element and q on last element.



p→next = NULL
free(q) → To free the memory!

Case 4: Delete the first node with a given value
This can be done exactly like Case 2 by bringing pointers p & q to appropriate positions

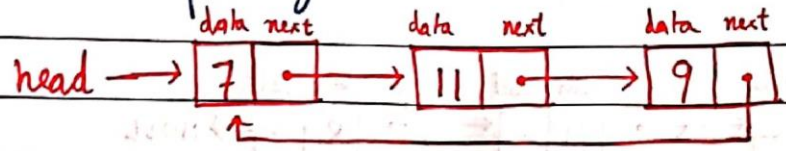


p→next = q→next;
free(q);

URBAN EDGE

Circular Linked List

A circular linked list is a linked list where the last element points to the first element (head) hence forming a circular chain



Operations on a circular linked List
operations on a circular linked lists can be performed exactly like a singly linked list.

Visit www.codewithharry.com for practice sets / code / more

Doubly Linked List

In a doubly linked list, each node contains a data part along with the two addresses, one for the previous node and the other one for the next node



Implementation

A doubly linked list can be implemented in C language as follows:

```
struct Node {
    int data;
    struct Node * next;
    struct Node * prev;
};
```

Operations on a Doubly Linked List

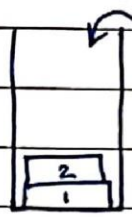
The insertion and deletion on a Doubly linked list can be performed by rewiring pointer connections just like we saw in a Singly linked list.

The difference here lies in the fact that we need to adjust two pointers (prev & next) instead of one (next) in the case of a Doubly linked list.

URBAN EDGE

Introduction to Stack Data Structure

Stack is a linear data structure. Operations on stack are performed in LIFO (Last in first out) order.



⇒ Item 2 which entered the basket last will be the first one to come out

LIFO (Last in first out)

Applications of Stack

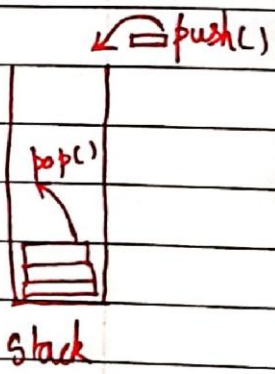
1. Used in function calls
2. Infix to postfix conversion (and other similar conversions)
3. Parenthesis matching & more...

Stack ADT

In order to create a stack we need a pointer to the topmost element along with other elements which are stored inside the stack.

Some of the operations of Stack ADT are:

1. push() → push an element into the stack
2. pop() → remove the topmost element from the stack
3. peek(index) → Value at a given position is returned
4. isEmpty() / isFull() → Determine whether the stack is empty or full.





Implementation

A stack is a collection of elements with certain operations following LIFO (Last in First out) dicipline.

A Stack can be implemented using an array or a linked list